

Лабораторна робота №4.

МСП-сервер, мультиагентна конфігурація та доказ користі

2 вересня 2026 р.

Мета

Опублікувати власний МСП-сервер над даними університету, під'єднати його до реального агента, порівняти одиночну й мультиагентну конфігурації на одному наборі задач і показати числами, чи виправдана друга архітектура.

Робота виконується **бригадою з трьох осіб**.

МСП SDK відкритий, модель локальна, трасування — через OpenTelemetry.

Склад бригади і розподіл ролей

Кожен учасник відповідає за свою підсистему, має власний результат і захищає його окремо. Спільним є лише інтеграційний крок і підсумковий звіт.

- **Учасник А — МСП-сервер і клієнт.** Пише сервер лише для читання з двома інструментами, тестує обидва рівні помилок, веде аудит-лог і під'єднує сервер до готового агента.
- **Учасник В — конфігурації агентів.** Збирає одиночного агента і supervisor із двома виконавцями на однакових інструментах та однаковій моделі, стежить за ізоляцією контексту й обліком токенів.
- **Учасник С — оцінювання й трасування.** Складає набір задач, рахує pass^k та інтервали, калібрує LLM-суддю і будує траси за конвенціями OpenTelemetry.

Інтеграційний крок виконують разом: сервер учасника А підключається до обох конфігурацій учасника В, після чого набір учасника С проганяється на кожній. Рішення «чи виправдана друга архітектура» ухвалює бригада за числами, а не за враженням.

Теоретичні відомості

Доки кожен застосунок пише власний конектор до кожної системи, кількість інтеграцій росте як добуток $M \cdot N$. Спільний протокол розриває добуток на суму $M + N$. Виграш з'являється не одразу: на малих числах суми й добутки майже збігаються, і накладні витрати протоколу помітніші за економію.

- **Підключення сервера є розширенням межі довіри, а не інтеграцією.** Сервер отримує ваш контекст і повертає текст, який стане для моделі інструкцією. Тому сервер лише для читання пишуть першим, а права додають після того, як запрацював журнал аудиту.

- **Протокол розділяє два рівні помилок.** Невідомий інструмент і аргументи, що не пройшли схему, — протокольні помилки JSON-RPC, корисні застосунку. Семантично хибні дані схему проходять і повертаються як результат із `isError: true`, придатний для самовиправлення моделі. Плутати рівні означає або ховати збій від застосунку, або позбавляти модель зворотного зв'язку.
- **У `stdio`-транспорті `stdout` належить протоколу.** Один `print()` у сервері ламає обмін мовчки — це найчастіша помилка перших годин роботи з MCP. Логи йдуть тільки в `stderr`.
- **Другий агент множить не лише якість.** Разом із ним ростуть канали координації, витрата токенів і кількість режимів відмови. Справжньою причиною виграшу є ізоляція контексту: якщо кожен агент тягне те саме, що бачать інші, витрата росте квадратично, а користі немає.
- **`pass@k` і `passk` рахуються на тій самій множині прогонів і дають протилежні відповіді.** Перша росте з k і описує дослідника, який має право перезапустити; друга спадає і описує продакшн, де спроба одна. Розрив між середньою точністю і надійністю задає *розподіл* успіху за задачами, а не середнє.
- **Інтервал Вілсона описує кожен частку окремо і не є тестом різниці.** Різницю оцінюють парно на тому самому наборі. Три прогони тих самих сорока задач не дають ста двадцяти незалежних спостережень: одиниця ресемплінгу — задача.
- **Суддя на основі моделі має виміряні упередження** — до позиції, до багатослівності й до власного стилю. Звітують не сиру частку збігів із людиною, а каппу: суддя, який завжди каже «А», має високу сиру узгодженість і нульову інформативність.

Корисні посилання для ознайомлення:

- Model Context Protocol: [з чого почати](#) та [Server: Tools](#);
- [Довідкові сервери](#) і [офіційний реєстр](#);
- [MCP Inspector](#) — довідковий клієнт для перевірки власного сервера;
- [Claude Code: MCP](#) — як підключити сервер до робочого агента;
- Anthropic, [How We Built Our Multi-Agent Research System](#) і Cognition, [Don't Build Multi-Agents](#);
- OpenTelemetry, [GenAI spans](#).

Порядок виконання

Позначка **[A]**, **[B]**, **[C]** вказує відповідального за крок; **[разом]** — крок виконує вся бригада.

1. **[разом] Визначити межу сервера:** які дані він віддає, які інструменти має і чого не має принципово. Рішення записується як контракт довіри.
2. **[A] Написати MCP-сервер лише для читання** над корпусом лабораторної 2 і підключити його як до власного клієнта, так і до готового агента — підійде MCP Inspector або будь-який відкритий клієнт, що працює з локальною моделлю. Кожен виклик має лишати слід в аудит-лозі.
3. **[A] Показати, що сервер розрізняє два рівні помилок** — протокольний і рівень виконання — і що кожен повертається у своїй формі.

4. **[В] Зібрати дві конфігурації** на однакових інструментах і однаковій моделі: одиночний агент і supervisor із двома виконавцями. Різнитися має рівно одна річ — топологія.
5. **[В] Виміряти координаційну вартість** другої архітектури: токени, передавання, час. Показати, скільки з цього припадає на службовий обмін.
6. **[С] Прогнати набір із 40 задач** у кількох повторах на обох конфігураціях і звести результат, надійність і ціну в одну таблицю.
7. **[С] Довести або спростувати різницю** довірчим інтервалом і відкалібрувати суддю на основі моделі на ручній розмітці.
8. **[разом] Оформити ноутбук.** Зібрати роботу в один Jupyter-ноутбук: код кожного учасника у своїх комірках із короткими коментарями. Обов'язково всередині: оголошення інструментів сервера, вивід тестів обох рівнів помилок, фрагмент аудит-логу, таблиця порівняння двох конфігурацій з інтервалом парної різниці, вартість координації в токенах, результат калібрування судді та приклад траси невдалої задачі. Наприкінці — явний висновок до 250 слів, чи виправдана мультиагентна конфігурація для вашої задачі.

Контрольні запитання

1. Як протокол перетворює $M \times N$ інтеграцій на $M + N$ і коли це не дає виграшу?
2. Чим ресурс відрізняється від інструмента в термінах ініціативи?
3. Чому один `print()` у сервері ламає stdio-транспорт?
4. Який клас помилки має повернути сервер на неіснуючу дату і чому саме такий?
5. Чому підключення MCP-сервера не є перевіркою його безпеки?
6. У чому різниця між `pass@k` і `passk`?
7. Конфігурація А розв'язала 31 задачу з 40, В — 33. Що з цього випливає?
8. Які три зсуви властиві судді на основі моделі й навіщо тут капша?
9. Які метрики фіксують координаційну вартість мультиагентної системи?
10. Що саме треба записати у трасу, щоб через тиждень відтворити невдалий прогін?
11. Чому один `print()` у стандартний вивід ламає stdio-транспорт MCP?

Література

1. Zheng L. та ін. [Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena](#). NeurIPS, 2023.
2. Cemri M. та ін. [Why Do Multi-Agent LLM Systems Fail?](#) arXiv:2503.13657, 2025.
3. Miller E. [Adding Error Bars to Evals](#). arXiv:2411.00640, 2024.
4. Yao S. та ін. [τ-bench: A Benchmark for Tool-Agent-User Interaction](#), 2024.